

| Introdução a Prolog

João Eduardo Montandon

DCC024

| Introdução

Prolog é uma linguagem de programação lógica.

- Um programa é composto por um conjunto de fatos e regras.
 - O programa é executado por meio de consultas.
 - A linguagem irá computar as possíveis saídas para a consulta.
-

```
parent(kim,holly).  
parent(margaret,kim).  
parent(margaret,kent).  
parent(esther,margaret).  
parent(herbert,margaret).  
parent(herbert,jean).
```

```
?- consult('hello.pl').  
true.
```

```
?- parent(kim,holly).  
true.
```

```
?- parent(margaret,holly).
```

```
false.
```

```
?- parent(X,holly).  
X = kim .
```

| Termos

Utilizados para construir as regras do programa.

1. Átomos.
2. Variáveis.
3. Termos compostos.

| Átomos

- Constantes: `123` , `-123` , `1.23` .
- Símbolos: `joao` , `*` , `.` , `=` , `[]` .
 - Devem começar com letra minúscula.

| Variáveis

Qualquer nome que comece com letra maiúscula.

`X` , `Joao` , `_` , `Joao_123` , etc.

| Termos Compostos

Um átomo seguido de uma sequência de termos entre parênteses.

```
atomo(termo1, termo2, ... , termoN)
```

```
x(y,z)
```

```
x(Y,Z)
```

```
parent(adam, seth)
```

```
parent(adam, Child)
```

⚠ **Isso não é uma função!** Pense como um dado estruturado.

Fatos

- Um programa em Prolog mantém uma base de fatos e regras de inferência.
 - Um fato é algo que é tido como verdade pelo programa, um *axioma*.
 - Sintaxe: termo seguido por um ponto: `parent(kim,holly).`
-

```
parent(kim,holly).
```

```
parent(margaret,kim).
```

```
parent(margaret,kent).
```

Fatos:

- `kim` é pai de `holly`.
 - `margaret` é mãe de `kim` e de `kent`.
-

Consultas

- Toda interação com o programa ocorre por meio de consultas.
- Para cada consulta, deve-se fornecer um predicado para o sistema.

- O sistema tentará "provar" a validade desse predicado.

-
- A "prova" se dá por meio do processo de **unificação**.
 - Dois termos são unificados se houver alguma ligação que os torne equivalentes.
 - O interpretador irá *unificar* o predicado com as regras existentes.
-

Qual o significado de cada consulta? 🤔

```
?- parent(kim,holly).  
?- parent(margaret, X).  
?- parent(holly, Y).  
?- parent(X,Y).
```

-
- Uma consulta pode conter uma lista de termos, separados por vírgula.
 - A linguagem unificará regras/fatos que atendam aos dois termos.
-

Qual o resultado dessa consulta?

```
parent(margaret, X), parent(X, holly).
```

Qual o escopo de **X**? 🤔

Quem são os avós de **kim**?

```
?- parent(P,kim), parent(GP, P).  
P = margaret,  
GP = esther ;  
P = margaret,  
GP = herbert ;  
false.
```

Use `;` para pedir mais provas à linguagem.

| Regras

- Linguagens disponibilizam abstrações para representar várias computações em um único comando.
 - Java: métodos.
 - ML: funções.
 - C: procedimentos.
 - Prolog: **regras de inferência**.

Uma regra permite combinar vários predicados e, com isso, gerar novos fatos.

```
grandparent(GP,GC) :- parent(GP,X), parent(X,GC).
```

O que diz essa regra?

```
?- grandparent(esther, X).  
X = kim ;  
X = kent.
```

Como definir uma regra para ancestral? 🤔

```
ancestor(X, Y) :- parent(X, Y).  
ancestor(X, Y) :- parent(Z, Y), ancestor(X, Z).
```

- Em Prolog, as regras são analisadas na ordem em que foram declaradas.
- Em chamadas recursivas, o caso-base deve ser fornecido **antes** dos passos recursivos.

| Entendendo Prolog

Como ler esta regra?

```
greatgrandparent(GGP, GGC) :-  
    parent(GGP, GP),  
    parent(GP, P),  
    parent(P, GGC).
```

Como uma receita:

Para provar `greatgrandparent(GGP, GGC)`, encontre algum `GP` e `P` para os quais podemos provar `parent(GGP, GP)`, `parent(GP, P)`, e então `parent(P, GGC)`.

Como uma declaração:

Para todo `GGP`, `GP`, `P`, e `GGC`, se `parent(GGP, GP)`, `parent(GP, P)`, e `parent(P, GGC)` são verdadeiros, então `greatgrandparent(GGP, GGC)`

também é.

Ambas são leituras válidas. Qual a diferença entre elas? 🤔

-
- A segunda leitura *não menciona a computação* realizada.
 - A computação está, na verdade, *embutida* no sistema da linguagem.
 - Isso permite a linguagem assumir aspecto declarativo!

| Operadores

Operadores são predicados já previamente implementados na linguagem.

```
?- parent(adam, seth) = parent(adam, X).
```

```
X = seth.
```

```
?- =(parent(adam, seth), parent(adam, X)).
```

```
X = seth.
```

Suporte aos operadores aritméticos tradicionais: $+$, $-$, $*$, $/$.

```
?- X+Y = 1+2*3.
```

```
X = 1,
```

```
Y = 2*3.
```

Percebeu algo de estranho com Y ? 🤔

- Em Prolog, os operadores não são avaliados automaticamente.
 - Lembre-se, o objetivo da consulta é **provar!**
-

| Listas

- Funcionamento similar a ML.
 - Permite armazenar termos.
 - `[]` representa lista vazia.
-

```
?- X = [1,2,3].  
X = [1,2,3].  
?- X = [1, joao, X, parent(ester, X)].  
X = [1, joao, X, parent(ester, X)].
```

Pattern-matching pode ser realizado com `|`.

```
?- [0,A,joao|B] = [0,1,joao,2,X,parent(kim,Y)].  
A = 1,  
B = [2, X, parent(kim, Y)].
```

| Predicados Flexíveis

1. `append`
 2. `member`
 3. `length`
-


```
?- append([1,2],[3,4], X).  
X = [1, 2, 3, 4].  
  
?- append(X, Y, [1,2,3,4]).  
X = [],  
Y = [1, 2, 3, 4] ;  
X = [1],  
Y = [2, 3, 4] ;  
X = [1, 2],  
Y = [3, 4] ;  
...
```

Como implementar o predicado `member(A,L)`? 🤔

```
member(A,[A|_]).  
member(A, [_,T]) :- member(A,T).
```

```
?- member(1, [1,2,3]).  
?- member(X, [1,2,3]).  
?- member(1, X).
```

```
?- length([1,2], X).  
X = 2.  
  
?- length(X, 2).  
X = [_, _].  
  
?- length([1,2,3|A], X).  
A = [],  
X = 3 ;
```

```
A = [_],  
X = 4 ;  
A = [_, _],  
X = 5 .
```

| Predicados Inflexíveis

Nem todos predicados funcionam de forma declarativa.

```
?- sort([2,3,1,4], X).  
X = [1, 2, 3, 4].  
  
?- sort(X, [1,2,3,4]).  
ERROR: Arguments are not sufficiently instantiated
```

Entenderemos mais a frente por quê.

| Negação Como Falha

- Representado pelo predicado `not`.
- Provar que *não* pode ser satisfeito != Negar o resultado da prova.

```
sibling(X,Y) :- parent(P, X), parent(P, Y).
```

Qual o problema com essa regra? 🤔

```
?- sibling(kim, kent).  
true.
```

```
?- sibling(kim, X).  
X = kim ;  
X = kent.
```

Como corrigir? 🤔

```
sibling(X,Y) :- not(X=Y), parent(P, X), parent(P, Y).
```



```
?- sibling(kim, kent). 👍  
true.  
?- sibling(kim, kim). 😊  
false.  
?- sibling(kim, X). 🙄  
false.
```

O que aconteceu?

$$\begin{aligned} sibling(kim, X) &\rightarrow not(kim = X) \wedge parent(P, kim) \wedge parent(P, X) \\ &\rightarrow not(kim = X) \wedge \dots \wedge \dots \\ &\rightarrow (kim = X) \rightarrow true \\ &\rightarrow false \wedge \dots \wedge \dots \end{aligned}$$

Como resolver? Colocar a verificação por último!

```
sibling(X,Y) :parent(P, X), parent(P, Y), not(X=Y).
```



```
?- sibling(kim, X).
```

```
X = kent. ✓
```

| Considerações Finais

A pergunta que fica é: *quando utilizar Prolog?*

- Em Prolog, lhe interessa as regras/restrições para o problema, mas não como ele será resolvido.
- Problema deve ser representado por meio dessas regras, a linguagem "faz" o restante.

Que tipo de problema se encaixa para esse caso? 🤔

Problema do fazendeiro, o lobo, a cabra e o repolho ^[1]

Um homem está viajando com um lobo, uma cabra e uma repolho. Ele chega à margem oeste de um rio e quer atravessá-lo. Há um pequeno barco disponível, mas ele é tão pequeno que o homem pode levar apenas uma de suas posses com ele no barco. Se ele deixar a cabra e o repolho sozinhos juntos, a cabra comerá o repolho. Se ele deixar o lobo e a cabra sozinhos juntos, o lobo comerá a cabra. Como o homem pode chegar à margem leste sem perder nenhuma de suas posses?

```
change(e,w).
```

```
change(w,e).
```

```

move([X,X,Goat,Cabbage],wolf,[Y,Y,Goat,Cabbage]) :-
change(X,Y).
move([X,Wolf,X,Cabbage],goat,[Y,Wolf,Y,Cabbage]) :-
change(X,Y).
move([X,Wolf,Goat,X],cabbage,[Y,Wolf,Goat,Y]) :-
change(X,Y).
move([X,Wolf,Goat,C],nothing,[Y,Wolf,Goat,C]) :-
change(X,Y).

oneEq(X,X,_).
oneEq(X,_,X).

safe([Man,Wolf,Goat,Cabbage]) :- oneEq(Man,Goat,Wolf),
oneEq(Man,Goat,Cabbage).

solution([e,e,e,e],[]).
solution(Config,[Move|Rest]) :-
move(Config,Move,NextConfig),
safe(NextConfig),
solution(NextConfig,Rest).

```

```

?- length(X), solution([w,w,w,w],X).
X = [goat, nothing, wolf, goat, cabbage, nothing, goat] .

```

1. https://pt.wikipedia.org/wiki/Problema_do_fazendeiro,_o_lobo,_o_carneiro_e_a_alface